Generation, Migration and Optimization of Cross-Language Static-Analysis Rules Based on Large Language Models

Zhanyi Hou, Zhiyu Duan, Minghao Yang, Mengdan Wu, and Shunkun Yang*
School of Reliability and Systems Engineering, Beihang University, Beijing 100191, China
houzhanyi61@buaa.edu.cn, duanzhiyu@buaa.edu.cn, yangminghao925@buaa.edu.cn,
mengdanwu@buaa.edu.cn, ysk@buaa.edu.cn
*corresponding author

2025 25th International Conference on Software Quality, Reliability and Security (QRS) | 978-1-6654-7771-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/QRS65678.2025.00015

Abstract—Rule-based static analysis tools are widely utilized for their high customizability. However, the creation of effective rules presents significant challenges, including the considerable human effort to handle the complexity of rules, and the additional costs involved in developing rules across various programming languages or frameworks. To address the significant challenges in manual rule creation for static analysis, this paper proposes a novel framework that leverages large language models (LLMs) to automate the generation of static analysis rules. The framework is specifically designed to alleviate the substantial human effort typically required in constructing and maintaining rule sets. Furthermore, we introduce a natural language-mediated rule migration methodology, which ensures semantic consistency when transferring functionally similar rules across different programming languages or frameworks. By seamlessly integrating LLMs with existing static analysis tools, our approach not only enhances the scalability and adaptability of rule generation but also enables efficient vulnerability scanning without the need for extensive computational resources such as large GPU clusters. This integration aims to bridge the gap between natural language understanding and program analysis, thereby facilitating more intelligent and resource-efficient static analysis. The method achieved 81.98% grammatical and 74.73% functional validity in rule generation for Semgrep, while migrating rules across 4 languages (Python, Java, JavaScript, and Golang). On the real-world engineering evaluation, it uncovered 9 unknown vulnerabilities in the latest version of the Linux Kernel undetectable by now.This work highlights the potential of our LLM-driven framework provides better handling of corner cases and maintains compatibility with industry-standard tools.

Keywords–Rule-Based Static Analysis; Large Language Model; Few-shot Learning; Rule Generation; Cross-Language Rule Migration; Rule Optimization;

1. Introduction

Static analysis at the source code level is widely applied in software development and quality assurance. This approach does not require compilation and execution of the code, so it is widely regarded as a rapid and economical method of identifying issues in their initial stages. To date, an increasing number of static code analysis tools have been introduced and seamlessly integrated into the overall quality assurance process. However, the effect of static analysis techniques is limited due to their inability to effectively handle project-specific static analysis [1][2].

Source-code level static analysis tools usually scan with a series of rules, known as ruleset. For example, a tool for C++ may have a ruleset containing Memory Leak Check, Array Out-Bounds Check, Opened File Without Closing Check, etc. The majority of static analysis tools usually integrate the ruleset with hard-coding logics, making them difficult to extend. To customize project-specific analysis, there are some frameworks for developers to create a specific checker to traverse through the program structure or the program flow, but it can be hardly possible for the developer who is not an expert in program analysis to develop this kind of custom checker even under a short-time training[3].

Rule-based static-analysis provides extensibility through customizable rules, allowing checks on project-specific business problems. The rule grammar is usually a kind of Domain Specific Language (DSL), which simplifies the process of customizing static analysis. For an example, Figure 1 is a rule to detect occurrences where a hardcoded password is utilized as a default parameter in a Python function for Semgrep, a popular open-source rule-based static analysis tool that supports checks on multiple programming languages. It is clear that this rule is defined in a straightforward YAML file, incorporating patterns alike those in the target programming language. Consequently, there is no need for us to manually write a specific checker performing code traversal or program flow analyses. Due to the simplicity, existing research has demonstrated that they can significantly enhance feedback on software quality [4]. However, despite writing rules is more simple than writing a checker, only up to 25% of maintainers are able to create rules effectively within a short-time training [3]. This is due to the complex nature of the rules, often involving multiple layers of nested patterns, presenting a steep learning curve for the rule writers. Besides, it is difficult for the manually written rules to fully consider and overcome the corner cases, which may lead to false-negatives and false-positives. Consequently, Beller's research revealed that fewer than 5% of the static analysis rules were employed in open-source projects are custom rules [1].

The extensive application of Large Language Models (LLMs) has greatly changed the realm of static analysis. In this aspect,

2693-9177/25/$31.00 ©2025 IEEE
DOI 10.1109/QRS65678.2025.00015

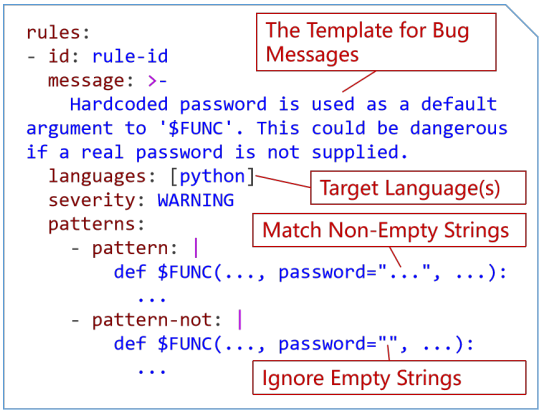

Figure 1. A Semgrep rule designed to detect hardcoded password in default argument for Python. When checking this rule, Semgrep selects all source files from the project according to the *languages* option, then match the code patterns specified by the *patterns* option. Once the code pattern is matched, a bug report will be raised according to the *message* and *severity* option. Note that the ellipsis symbol (...) stands for *any* statements or expressions.

the majority of approaches employ LLMs to comprehend source code or the data produced by static-analysis tools, thereby determining the presence of bugs, as well as their type and location. However, these methods depend on the continuous inference from LLMs during code inspections, making them extremely expensive for development teams with constrained computational resources. Yang presented KNighter, which has demonstrated a paradigm-shift by utilizing LLMs to generate a new project-specific code checker for C/C++ projects based on Clang Static Analysis framework, and proved that the generated checker can effectively scan code at a significantly reduced cost [5]. Nonetheless, this method is oriented to specific programming language to our knowledge, and the prompting methodology for the LLM must be reconstructed when adapting this approach to different programming languages.

To sum up, the field of rule-based static analysis faces the following problems: (1)The complexity as the nature of rules, (2)the vast computational resource consumption, and (3)the difficulties of migrating rules to other languages.

To address these problems above, we propose a method based on LLMs that generates static analysis rules from the pattern of historical code repairs, utilizing grammar rule prompts and few-shot learning in context. Additionally, our approach supports the migration of existing static analysis rules written for a specific language to other languages. It also supports the generation of test samples for existing rules through the code generation capabilities of LLMs, thereby potentially exposing potential false positives or negatives in corner cases to effectively optimize existing rules.

The contribution of our work is:

- A framework based on LLMs for software static analysis rules is proposed to address the limitations of manually extracted rules that are difficult to consider corner cases. Utilizing few-shot learning, this framework leverages the domain knowledge acquired by LLMs during unsupervised pre-training, integrating syntax rule specifications, historical error information, and rule description optimization to guide the generation behavior of LLMs. This effectively solves corner cases that are difficult for humans to consider, reducing false negatives and false positives in rule analysis results.
- A cross-language rule transfer method based on natural language mediation is proposed to tackle the high resource consumption of LLMs and improve the rule migratability. LLMs are used to translate rules from the current language into natural language descriptions of rules in the target language, generating new rules in the target language while ensuring the consistency of rule contents.
- Our proposed method simultaneously utilizes the performance of both LLMs and static analysis tools to reduce the inference burden of LLMs, avoiding dependence on large-scale GPU clusters. This makes the code scanning process more resource-efficient.

Our experimental results demonstrate the efficacy of the proposed method across multiple domains. On Semgrep's official ruleset, the approach achieved 81.98% grammatical validity in rule generation, with 74.73% of these being functionally valid. For cross-language rule migration, it successfully converted 18 original rules into 48 target-language variants, maintaining 79.17% grammatical and 84.21% functional validity, while additionally identifying 7 optimization points in Semgrep's official ruleset and corrected 5 of them. When evaluated on the dataset of Linux Kernel containing 61 bug fixes, the method generated 41 effective rules - surpassing the state-of-the-art method KNighter's performance (37 rules) - and discovered 9 previously undetected vulnerabilities that eluded conventional static analysis tools.

2. Background

2.1 Rule-Based Static Analysis

Currently, many software development teams have integrated software static analysis tools into their program analysis processes. As software projects evolve, traditional static analysis tools have gradually revealed deficiencies in scalability, with their matching logic usually implemented through hard coding. Although some tools support user-defined rules, they often only allow for simple matching rules based on regular expressions [6][7]. A highly customizable approach is to use lower-level code analysis frameworks to tailor their own code analyzers, such as Clang, CodeQL and Joern. However, these tools typically depend on specific programming languages or corresponding APIs, requiring code quality control personnel who define detection rules to have strong domain knowledge [5][8].

In recent years, rule-based code static analysis tools have been introduced. These tools use syntax similar to the analysis-target language to define matching patterns, without involving

domain knowledge such as Abstract Syntax Trees(AST), and support more complex pattern-matching. The widely used rule-based code static analysis tools mainly include Comby and Semgrep. Comby is a lightweight static scanning tool based on simple rule pattern matching, including expressions, statements, and simple nested structures. Besides, Semgrep is a static analysis tool that supports complex structures and basic dataflow checks, enabling code inspection without execution to detect security vulnerabilities, quality issues, and compliance violations. Its YAML-based rule grammar mimics the target language's syntax, significantly reducing the learning curve compared to tools like CodeQL. By converting code into a Universal Abstract Syntax Tree (UAST), Semgrep applies uniform pattern-matching across 30+ languages (e.g., C/C++, Python, Java), while its active community and rule-sharing platform enhance adaptability and collaborative rule development.

2.2 The application of LLMs in software static analysis

The latest advancements in LLMs have provided a flexible way to avoid the typical challenges of traditional software static analysis techniques, offering a promising alternative for understanding and summarizing code behavior [9].

LLMs possess strong capabilities in coding and are widely used for generating code in various programming languages. Additionally, LLMs also support the generation of configuration files, as demonstrated by existing research that shows the automated generation of network configuration items through LLMs [10][11][12]. For languages not included in the training samples of LLMs, research by Wang et al. has proven that providing formal syntax specifications in Backus-Naur Form (BNF) in the context can make LLMs competitive in various DSL generation tasks [13]. Existing research has also demonstrated that learning based on prompt contexts can effectively generate DSL on existing LLMs [14][15].

Furthermore, LLMs can process diverse and flexible input data formats, including natural language and source code [9], effectively handling complex code snippets and producing intuitive and readable understandings. For instance, they can extract program invariants that are difficult for traditional program analysis methods to extract [16] [17]. They can also summarize complex functions or features into simplified versions of programs, reducing the scale of program analysis verification [18]; Li confirms that LLMs can identify path conditions and control flow structures in code, thereby inferring different execution paths, even in the presence of loops [19]. Yang et al.'s work demonstrates that LLMs can effectively extract code change patterns [5]. In summary, while traditional static analysis methods provide formalized and exhaustive analysis, LLMs can offer more intuitive, human-like understanding and serve as an important complement to static analysis methods when traditional methods are insufficient.

In this paper, our research primarily utilizes the text generation and code generation capabilities of LLMs. The text generation capability is mainly used to generate descriptive documents based on code changes, while the code generation capability is primarily manifested in using LLMs to generate static analysis rules and code snippets for testing rules.

- **In generating rule description documents based on code changes**: Previous studies have demonstrated that LLMs can effectively compare code before and after changes, extracting corresponding code patterns [Knighter]. In this paper, we propose to use this method, leveraging LLMs to compare code changes and generate the required rule description documents.
- **In the generation of rules**: Since static analysis rules are typically carried in popular configuration file formats such as YAML, XML, etc., and each field has specific requirements and constraints, such as the necessity of having fields like "rule name," "rule error message," and "fault pattern to be matched," they can be regarded as a DSL for describing pattern matching rules in software source code. To meet the demand for generating such DSLs, we have added corresponding grammar rules to the generation process, supplemented by a few examples. However, common rule-based static analyzers' official documentation usually does not provide formal rule grammar descriptions like BNF but only offers natural language descriptions and a few small-scale examples. Therefore, prompting LLMs to generate static analysis rules based on such grammar documentation and few-shot learning can be challenging.
- **In generating code snippets for testing rules**: the primary approach used is the code generation capability of LLMs. Since the LLMs are usually well-trained on common programming languages and the requirement during training is mainly the correctness of the generated code snippets rather than exhaustively trying various plausible grammars to explore various corner cases, it can be challenging for these models to produce code snippets that reveal deficiencies in existing rules.

Aiming at the above challenges, this paper conducted quantitative research respectively, and the results can be seen in Research Questions (§ 5.2).

3. Method Design

Our proposed method works as a LLM agent to generate, migrate and optimize static-analyzer rules, as illustrated in the Figure 2, 5 and 6 respectively.

The workflow mainly consists of three stages: The Generation, Migration and Optimization of Rules. In the first stage, our method recognizes the code patches and generates the description of rules, then generates the static-analysis rule. The generated rules are evaluated on the test snippets to verify theirs validity, and then the rule descriptions can be optimized according to the false positives and false negatives. In the second stage, our method generates natural-language descriptions and corresponding test snippets for the migrated rule, then creates the rule subsequently. In the last stage, our method reads an existing rule, no matter whether its manually written or generated, and creates test snippets for more corner cases to detect probable existing false positives

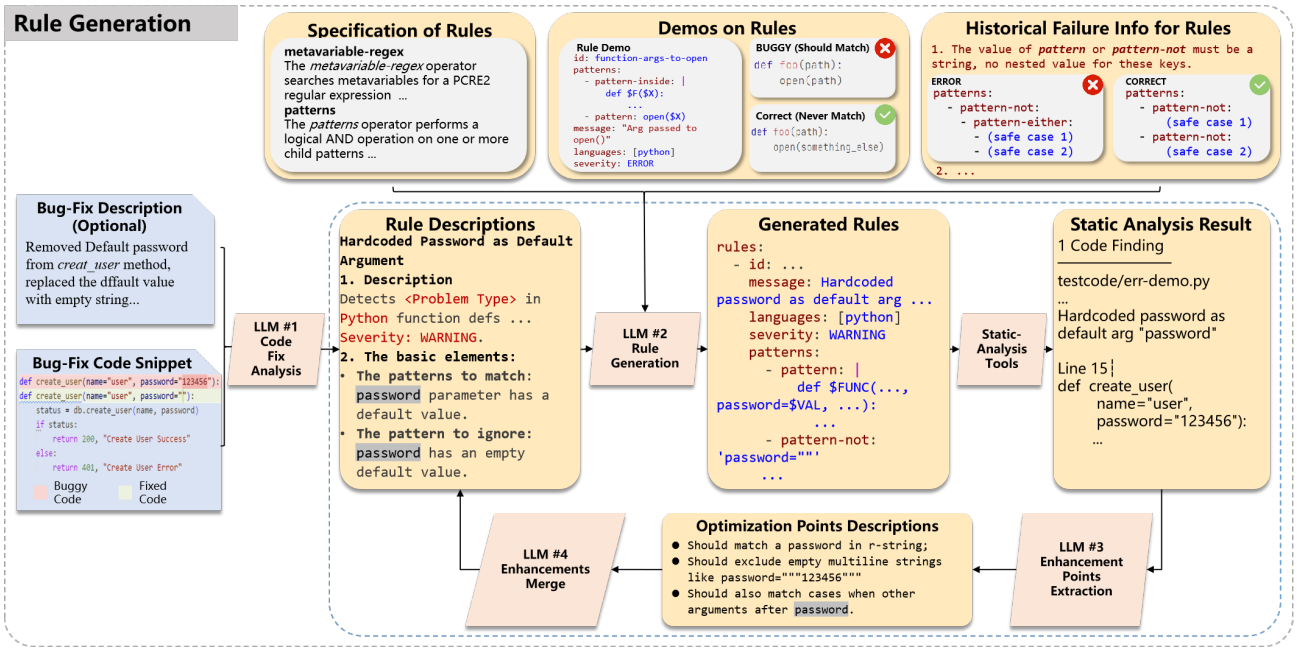

Figure 2. The Workflow of rule generation from code patches

or false negatives, then following the same steps for rule enhancement in stage 1.

3.1 Rule Generation From Code Patches

3.1.1 Generation of Rules Description

This step aims to generate descriptions of rules from the bug-fixing code patches. Initially, the buggy and correct code are extracted from the bug-fix submission, and then the prompts that encompass task requirements along with both buggy and correct code will be constructed, shown as Figure 3. The buggy and correct cases consist not only of the single-line statement that the bug appears but also of the entire function containing the statement and its correct counterpart, thereby providing more abundant information for the *LLM #1* to generate the description.

The generated descriptions are anticipated to include a basic description of the rules, encompassing the function, target language, and severity. Additionally, they may also incorporate the fundamental elements necessary for synthesizing the rule, such as patterns to be matched and those to be ignored. An example of generated descriptions is shown in Figure 4.

3.1.2 Generation of the Static-Analysis Rules

This step aims to generate the static analysis rules corresponding to the descriptions. Due to the limited samples (about 1k), the grammar prompting was adopted rather than fine-tuning. To optimize the accuracy of rule generation by the LLM agent, additional input including the grammar of static analysis rules in natural language and small examples was provided to the *LLM #2*. Moreover, after generating a certain number of rules and evaluating them, we may identify common error-prone

```
As an experienced expert in the field of static analysis,
please help me compare the correct and incorrect code in
the following and develop a description of the elements
that the static analysis rules need to match.
To locate the problematic code and ignore the irrelevant
parts in the problematic function, you need to analyze the
code structure, identify the key elements, and then extract
the corresponding code patterns that need to be matched for
static analysis.

# Buggy Code:
```python
def whoops(password="this-could-be-bad"):
    print(password)
```

Correct Code:
```python
def say_something_else(something_else="something else"):
    print(something_else)
def say_something(something):
    print(something)
def ok(password=""):
    print(password)
```
```

Figure 3. A prompt containing the buggy code (before fix) and correct code (fixed) for generating the description of rule

points in the historically generated rules. This information is also inputted into the *LLM #2*.

3.1.3 Verification and Enhancement of the Generated Rules

The purpose of this step is to verify and enhance the generated rules to make sure they produce the expected result. Since the prompts used to generate the rules are automatically generated by a LLM, they may lack some necessary information for rule generation and fail to constrain the generation behavior of the LLM, then resulting in false positives or false negatives.

We optimize the prompts based on the output generated by the rules, targeting situations of false negatives or false positives.

```
# Hardcoded Password as Default Argument
## Description
This rule detects when a hardcoded password is
used as a default argument in Python function
definitions. This is a security risk because it
exposes credentials if the default value is
accidentally used in production environments.
Severity: WARNING.
## The basic elements of the rule:
1. The pattern to match:
   Function definitions where the `password`
parameter has a non-empty default value (e.g.,
`password="secret123"`).
2. The pattern to ignore:
   Function definitions where the `password`
parameter has an empty default value (e.g.,
`password=""`).
```

Figure 4. The generated rule description for the rule detecting *hardcoded password in default arguments*

This work is primarily done by the LLM but requires some human intervention.

First, we compare the reported errors with the expected errors. We assume the reported errors are a set S_R and the expected errors are a set S_{Ex}. The difference set of S_R and S_{Ex} stands for the reported bugs that no actual bug exists (false positives), denoted as S_{FP}. Likewise, the difference set of S_{Ex} and S_R is the actual bugs that are not reported (false negatives), denoted as S_{FN}.

Each case in S_{FN} or S_{FP} is used as a prompt to ask the *LLM #3*, "What are the points that are missing in the rules?". Then the LLM will generate the description of enhancement points, mainly including the corresponding missing points. For example, in Figure 2, the LLM can identify "Possible failure to match strings starting with 'r'". Then another *LLM #4* is used to integrate these contents into the original prompts, after which the rule generation process is executed again. In this experiment, the enhancement process is conducted for a maximum of 3 rounds. The generated rule could be effective in recognizing both the erroneous and correct instances derived from the bug fix discussed in section 3.1.1. However, it may not adequately handle corner cases. To address this, employing the technique outlined in section 3.1.3 allows for the optimization of the rule, thereby enhancing its capability to manage these corner cases effectively.

3.2 Rule Migration Based on Natural Language Media

This phase aims to migrate an existing rule on a specific language to another language or programming framework. The prompt of this process is an existing rule that can be human-written or generated, and a consistent migration prompt for all migrations, with the sole variation being the target language. Note that real bug-fix patches are not needed in this step because the test snippets are generated simultaneously by LLMs.

We add the content of the existing rule to the corresponding template (see Figure 2) and include a common prompt context (containing information about the grammar of the rules, rule examples, and historical errors in generating rules, etc.), which can then be used to generate the description of the transplanted rule and the accompanying test code through a LLM.

In this step, the first LLM instance can generate rule descriptions based on the prompts. Since the programming language to which the rule is being ported often does not have corresponding test code, this LLM instance will also generate such code simultaneously.

The next LLM (Large Language Model) instance will generate the transplanted rule based on the description of the rule, followed by verification and enhancement of the rule, which follows the same process as described in §3.1.3.

The description of the transplanted rule will be verified and optimized on the test code after transplantation, following the same process as described in §3.1.3.

3.3 Rule Optimization Based on Corner Cases Exploration

The input of the rule optimization process can be either a generated rule or a previously manually defined rule. We input the generated rule, common prompt context (including information about the grammar of the rules, rule examples, and historical errors in generating rules, etc.), and the prompts for rule optimization into a LLM. In this step, the LLM instance can generate additional test code snippets (a term that needs to be thought of to represent this kind of code) for the input rule based on these prompts, thereby covering more scenarios. Subsequently, static analysis tools are running on these code snippets to check if there are any false negatives or false positives on the newly added code. If there are new false positives and false negatives, this information is used to guide the optimization process of the rule, following the same process as described in §3.1.3.

4. Implementation

4.1 Input Data Collection

Dataset for Rules Generation: For the input data for rules generation, we first randomly selected 111 rules out of the official ruleset of Semgrep, each rule contains a rule YAML file and code snippets for buggy/correct code cases. Based on the collected buggy/correct code corresponding to each rule, we constructed a bug fix for each rule, containing the buggy code and the correct code, and then subsequently used these code changes as input for the rule-description generation process. After the descriptions were generated, we manually checked the descriptions to ensure they were corresponding to the original rules. The selected rules and the quantity of different programming languages are shown in Table 1.

Grammar Prompt and Few-Shot Examples: We used documentation from Semgrep's official site, including parts of *Rule Syntax* and *Pattern Syntax*. We converted the text of the official site into Markdown, then inserted the grammar into the LLM chat context. As the documentation was intended for human readers, there were no formal specifications for the rulesets, and each pattern specification item was usually followed by a small code snippet example, we used them as few-shot examples.

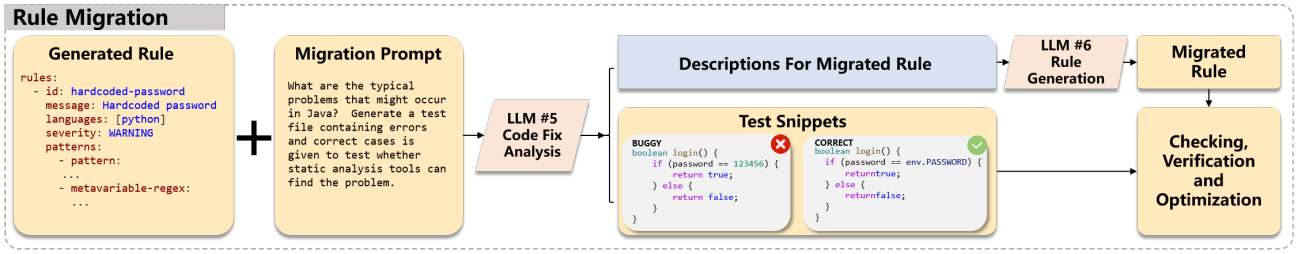

Figure 5. The Workflow of rule migration based on natural language media

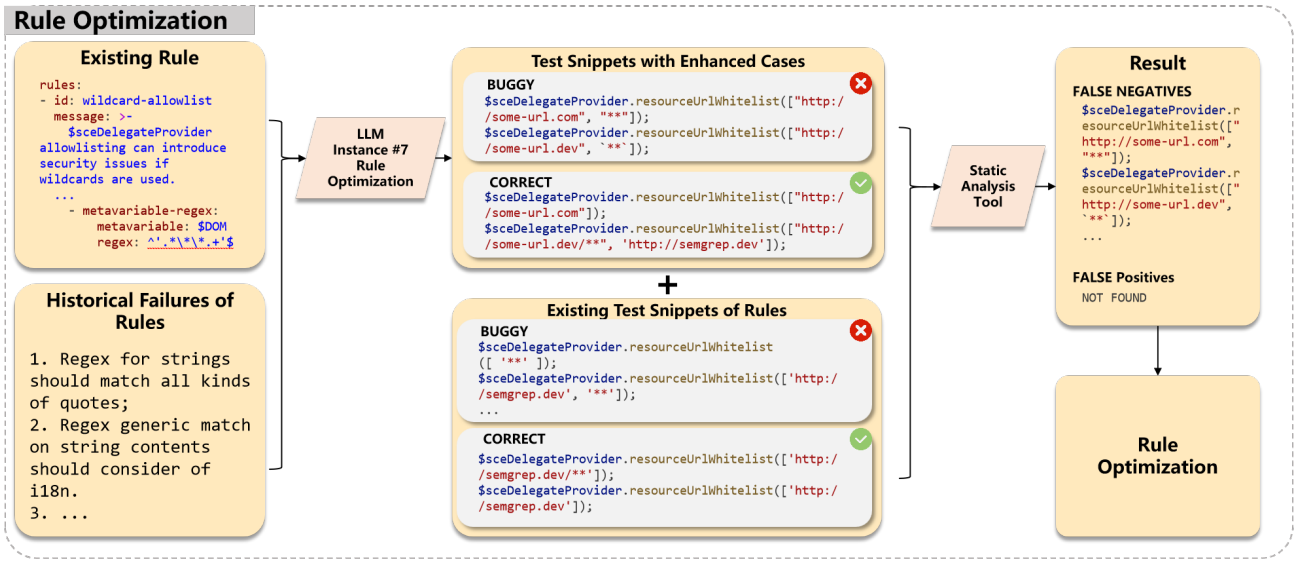

Figure 6. The workflow of rule optimization based on corner cases exploration

Table 1. The selected rules and the quantity of different programming languages

Index	Programming Language	Rules Count
1	Python	48
2	Java	20
3	JavaScript	18
4	Ruby	9
5	OCaml	9
5	C/C++	7

4.2 Static Analysis Tools Setup

In this paper, we chose Semgrep v1.110.0 as the static analysis basis. When validating the generated rules, we selected the corresponding official rule to scan the same test code. The bugs obtained from the official rule are considered as the actual bug set S_{Ex}. After scanning with the generated rules, if Semgrep's output indicates that the rule is valid in grammar, we proceed to the next step; otherwise, it is directly marked as "grammar invalid". For grammar-valid rules, the set of bugs is denoted as S_R. By performing difference operations between S_R and S_{Ex}, we can obtain the false negative set S_{FN} and the false positive set S_{FP} as §3.1.3 described.

4.3 Large Language Models Setup

In this experiment, tasks such as code information extraction and rule generation require strong reasoning abilities. Therefore, we chose the open-source LLM DeepSeek-R1 as the base for each LLM instance, for its low cost and strong reasoning capabilities which made it suitable for tasks that demand a certain level of logical reasoning [20].

When invoking DeepSeek-R1, the main hyperparameter to configure is $temperature$. The hyperparameter $temperature$ can be set between 0 and 2, controlling randomness and reasoning ability. In tasks involving code or rule generation (*LLM#1, #3, #4*), the analysis should be as non-random as possible, hence requiring a lower value and we set $temperature = 0$. For other LLM instances, as they are performing natural language generation tasks, too low $temperature$ might lead to oversimplified outputs. Therefore, we set the $temperature = 1.0$.

5. Experiment Setup

5.1 Definitions

Grammar-Validity: Grammar-valid rules are defined as the generated rule that is recognizable to the static analysis tool.

Table 2. Contributions of different prompt methods

Index	**Prompt Method**	**Valid Rules**	**Complete Rules**	**Valid Rules Proportion**	**Complete Rules Proportion**
1	Baseline	68	38	61.26%	55.82%
2	Grammar Info	81	46	72.97%	56.79%
3	Grammar Info + Historical Errors	92	51	82.88%	55.43%
4	Grammar Info + Rule Description Enhancement	81	61	72.97%	75.31%
5	Grammar Info + Historical Errors + Rule Description Enhancement	91	68	81.98%	74.73%

To evaluate the validity of grammar, we applied every rule to the relative code snippet, and then examined whether the tool reports rule grammar error or not.

Functional-Validity: Functional-Valid rules are the generated rules that can report bugs in the buggy code snippet.

For any buggy code snippet containing N bugs which typed the same as the rule, if the developed rule produces B_{TP} True Positive bug reports, and B_{FP} False Positive bug reports, we define these two indicators to evaluate the validity of generated rule:

$$False\ Positive\ Rate: FP = \frac{B_{FP}}{N} \tag{1}$$

$$True\ Positive\ Rate: TP = \frac{B_{TP}}{N} \tag{2}$$

Complete Rule: In particular, if the generated rule achieves $FP = 0$ and $TP = 0$ on the input code snippets, we call this rule Complete Rule.

5.2 Research Questions

5.2.1 RQ1: What Are the Contributions of Different Prompt Mechanisms?

The contributions of different prompt mechanisms are shown in Table 2. Note that the *Baseline* means the prompt only with rule description.

- Baseline: For the baseline, this method generated 68 valid rules (61.26%) and 38 complete rules (55.82%), indicating relatively low effectiveness.
- Grammar Info: After introducing grammar information, the number of valid rules increased to 81 (72.97%), and the number of complete rules increased to 46 (56.79%). This shows that grammar information promotes completeness and effectiveness of rule generation, and may not contribute to the functional validity of rules.
- Grammar Info + Historical Errors: Combining with historical error analysis, the number of valid rules further increased to 92 (82.88%), but the proportion of complete rules is also at about 55%. This suggests that historical error points primarily improve grammar validity, but cannot increase functional validity either.
- Grammar Info + Rule Description Enhancement: Upon the introduction of rule description enhancement, the number of valid rules decreased to 81 (72.97%), but the number of complete rules significantly increased to 61 (75.31%). This indicates that rule description enhancements are more beneficial for the functional validity of rules, while the grammar validity remains unchanged.
- Grammar Info + Historical Errors + Rule Description Enhancement: The final method maintained high effectiveness (91 rules, 81.98%) while achieving 68 complete rules (74.73%), demonstrating that the combination of prompt methods could improve both grammar and functional validity of rules effectively.

From the results above, we can find that the grammar information and historical errors are critical for enhancing rule effectiveness, while rule description enhancement contributes the most to completeness.

5.2.2 RQ2: What Is the Effect of Rule-Generation on Real-World Commits and Comparing to Other Detection Methods?

To compare the effect of our method on rule generation, we chose the SOTA work KNighter for comparison. As KNighter provided a dataset publicly available encompassing 61 code commits for bug fixes in Linux Kernel, we evaluated our method on this dataset in order to not only compare with the SOTA method but also evaluate its performance on Linux Kernel.

Each commit in the dataset includes both a faulty version of the code and a corrected version. We carried out the experiment on this dataset by our method, and then evaluated the rules by manually checking the rules to tell if they are valid in grammar or in functionality. Using our method, we conducted experiments on this dataset and subsequently evaluated the extracted rules through manual inspection.

These results demonstrate that our method generated 41 valid rules compared to KNighther's 37 rules, modestly outperformed KNighter, while our method needs more refining compared to KNighter.

Following our experiment on generating rules from Linux Kernel bug fixes, we scanned the most recent version of the Linux Kernel (version aef17cb3d3c4) that we could access when writing this paper. After scanning and confirming the

Table 3. Effect of Rule-Generation on Linux-Kernel Bug-Fix Commits

Index	Bug Type	Total	Our Method				KNighter			
			Invalid	Direct	Refined	Fail	Invalid	Direct	Refined	Fail
1	Null Pointer Deref	6	0	2	3	1	1	2	2	1
2	Integer Overflow	7	1	2	3	1	3	1	3	0
3	Out-of-Bound	6	1	3	2	0	2	4	0	0
4	Buffer Overflow	5	2	1	2	0	3	2	0	0
5	Memory Leak	5	0	1	3	1	2	3	0	0
6	Use After Free	7	1	3	3	0	4	2	1	0
7	Double Free	8	4	1	2	1	1	5	1	1
8	Use Before Init	5	2	3	0	0	1	1	3	0
9	Concurrency	5	2	1	1	1	2	3	0	0
10	Misuse	7	1	5	0	1	3	3	1	0
11	Total	61	14	22	19	6	22	26	11	2

results with corresponding maintainers, we finally found 9 valid defects by 2 rules, shown in Figure 7.

```
// FILE: drivers/net/wireless/mediatek/mt76/mt7915/mcu.c
// INFO: Missing Non-Null Judgement on vc before dereference.
// FROM: Commit f503ae90c7355e8506e68498fe84c1357894cd5b
const struct ieee80211_sta_he_cap *vc =
    mt76_connac_get_he_phy_cap(phy->mt76, vif);
const struct ieee80211_he_cap_elem *ve = &vc->he_cap_elem;

// FILE: drivers/gpu/drm/amd/amdgpu/amdgpu_userq_fence.c
// INFO: Allocating an array-memory with memdup_user and
//       multiplication with probable unsanitized length.
// FROM: Commit 3e91a38de1dce97b385d732a5f444264ea8fbd92
num_syncobj = wait_info->num_syncobj_handles;
// data is a void* argument of function passed-in externally
struct drm_amdgpu_userq_wait *wait_info = data;
syncobj_handles = memdup_user(
    u64_to_user_ptr(wait_info->syncobj_handles),
    sizeof(u32) * num_points);
```

Figure 7. The two types of bugs that found in Linux Kernel. Note that the property *FROM* is the the commit from which our method extracted the checking rule.

We have also compared the performance of our method to common static-analysis tools or rulesets including Smatch, CPPCheck, Semgrep's Official Rulesets and TScanCode, on a Ubuntu 22.04 server with 64GB Memory. After the running of tools on the same code version of the Linux Kernel, Smatch found 1725 errors and 2417 warnings, CPPCheck found 11036 errors and 367 warnings, Semgrep's official ruleset generated totally 22704 alarms, while TScanCode abnormally quit with no reports generated though we have tried it on both Linux and Windows for multiple times. None of the static tools discovered the unknown defects that our method revealed, which may be due to the lackings in domain knowledge of specific codebases that LLM-based methods could gain from code and defect descriptions.

5.2.3 RQ3: Could Generated Rules Be Generalized for Different Languages or Frameworks?

Rule migration refers to the process of transferring existing Semgrep rules and their descriptions to other programming languages or different development frameworks within the same programming language using Large Language Models (LLMs).

During the rule migration process, 16 original rules, covering Python, Java, and JavaScript, were migrated. A total of 48 new rules were generated, encompassing Python, Java, JavaScript, and Golang.

We chose 18 typical rules for the migration attempt, with each rule being transferred to between 1 and 4 new rules, amounting to a total of 48 new rules. Upon verification, 38 rules (79.17%) were found to be effective in grammar. Among the grammar-effective rules, 32 rules (84.21%) behaved as expected.

In the experiment, we have found some types of rules that are unlikely to be migrated, listed below:

- Rules that are specific to version compatibility within the same language or library and are meaningless for other languages. Note that this does not include compatibility rules for encryption algorithms or communication protocol versions.
- Rules that are strongly related to the syntax of the target language, such as the dangerous-annotations-usage rule involving type hint annotations in Python, which is almost impossible to migrate.

5.2.4 RQ4: What is the Effect of Our Method Optimize Existing Rules?

During the experimental process, our method identified 7 issues in the official rules, and could fix 5 of them in total. The spotted issues mainly fall into the following three categories:

- Errors in regular expressions. When matching Python/JS strings using regular expressions, only strings enclosed in double quotes were considered, leading to the omission of strings enclosed in single quotes (Python, JS) or backticks (JS). We spotted 3 problematic rules of this kind, and our proposed method generated a fix for all these rules.
- Potential omission of certain cases in selective matching. We have found 2 problematic rules of this kind, and our method could fix them. For instance, in Figure 8, Rule *dangerous-annotation-usage* matches cases in Python that a dynamic type annotation added to a class property must be a built-in

type. Our method found that the official rule does not match built-in variable type *type* and *bool*, and proposed a feasible fix by adding two missing types.

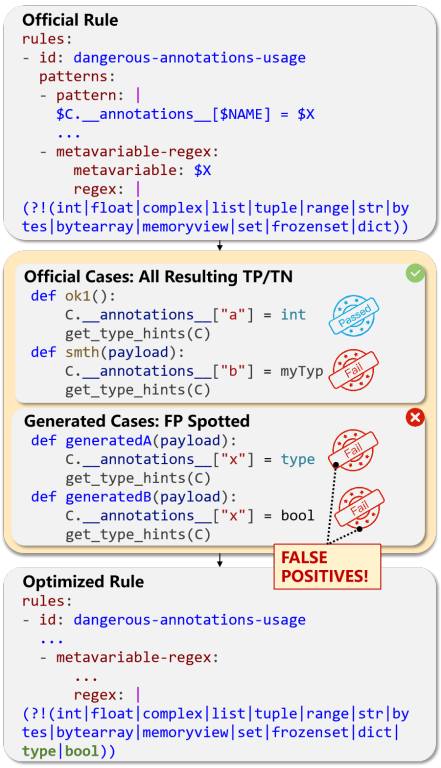

Figure 8. A false-positive case spotted in the official ruleset of Semgrep that missed to match built-in types ”type” and ”bool” in the rule.

- Failure to consider internationalized text content when matching Unicode strings. We have spotted 2 problematic rules of this type, but we failed to prompt the LLM to generate the correct regex expression after three times of enhancement iterations. An example is shown in Figure 9, the rule ”text not internationalized” should detect languages other than English.

6. Related Work

6.1 Large Language Models (LLM) In Software Static Analysis

Compared to traditional deep learning models, LLMs have demonstrated exceptional capabilities in understanding code semantics, capturing deep semantic information [21][22]. Unlike methods based on classic deep learning models, Large Language Models (LLMs) can directly accept natural language inputs, eliminating the need for complex preprocessing and feature extraction of the input code and tool reports [22], facilitating portability across different programming languages and static analysis tools. Consequently, program static analysis methods based on LLMs have gained widespread application.

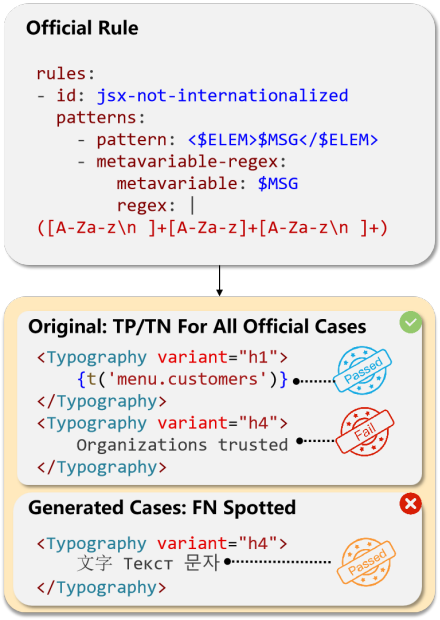

Figure 9. A false-positive case spotted in the official ruleset of Semgrep. Note that all of the three non-ASCII words in the false-positive case mean ”text”, and they are respectively in Chinese(Japanese), Russian and Korean.

In this paper, we categorize the commonly used static analysis methods based on LLMs into three main types, discussing them from the following three aspects: Direct Use of LLMs for Static Analysis: This approach primarily leverages the code comprehension capabilities of LLMs, combined with appropriate prompts, to analyze input code snippets, ultimately identifying defects, classifying them, and locating them. Methods like [23], [24], and [25] require fine-tuning of pre-trained models, whereas approaches like [26] and [27] use prompt engineering for predictions. This method fully utilizes the grammar and semantic knowledge within LLMs. However, due to the limited context length of these models, in practical software projects, it’s often necessary to extract program components closely related to the issue for analysis. Additionally, this method can incur significant computational costs.

Using LLMs to Further Analyze and Filter Reports from Traditional Analysis Tools: Traditional static analysis tools typically use pattern matching to detect issues in code, generating a large number of error messages with high false positive rates. Before the widespread adoption of LLMs, using classic deep learning models to further analyze and filter the outputs of one or more static analysis tools was proven to significantly enhance accuracy, especially in reducing false positives [28][29]. After the widespread adoption of LLMs, this method has been extensively applied with remarkable results. For instance, [19] further analyzed and filtered the output of a state-of-the-art uninitialized variables detection tool UBITect,

combining static analysis and symbolic execution, effectively reducing false positives and improving recall. [30] proposed an approach intertwining static analyzer calls with LLM queries, analyzing error handling statements in typical programs, and using LLM query results for subsequent analysis when static analysis stalls. Compared to directly using LLMs, this method is usually limited to analyzing the problem locations reported by static analysis tools, thus saving computational resources. However, its limitation is its dependence on the capabilities of the upstream static analysis tools; if these tools have weak detection capabilities for the target language, producing only a few reports, the information provided to downstream LLMs is severely limited, potentially failing to effectively identify issues.

Using LLMs to Generate New Static Analysis Tools: This approach is based on the code generation capabilities of LLMs, writing new static analysis tools for specific issues using program analysis libraries. A representative work is [5] proposed by Yang et al. in 2025, which uses LLMs to generate C++ code based on the Clang Static Analyzer (CSA), constructing static detectors for C/C++ programs. Experiments have shown that this method can effectively scan the Linux Kernel codebase. The main advantages of this method include avoiding the high cost and context limitations of directly using LLMs for large codebases, and offering strong scalability. Moreover, the detectors can naturally evolve incrementally with Kernel development without continuous LLM involvement. Lastly, the product of this method is the source code of the static analysis detector, transparently and human-readably encoding and detecting bug patterns, enhancing traceability and explainability, and helping overcome model hallucination issues. However, manually inspecting the code analyzers generated by LLMs in this method requires extensive knowledge of static analysis, posing high demands on personnel, and the method proposed in this paper can effectively reduce the reliance on domain expertise.

6.2 Automatic Generation of Software Static Analysis Rules

Static analysis techniques can efficiently uncover software defects, but the rules of most popular static analysis tools usually only support general bug patterns without specific target software projects, such as checks for reading and writing memory, pointers, file descriptors, and division by zero checks. Researchers like Johnson have previously pointed out that the lack of customizability for actual projects is one of the reasons SA tools are rarely used [31]. Ray and others have found that developers often make changes with similar patterns in their software, known as project-specific bug patterns (PSBP), which can be extracted through a process of mining existing code defects and their repair patterns, thus matching and detecting new versions of software or software modules where defects have not yet been exposed [32].

Higo and others have proposed a method based on comparing changed code text, extracting PSBP for specific software projects by contrasting code changes during software fault repairs [33]. Chen and others have introduced the VulChecker method, which incorporates natural language processing mechanisms to detect semantically relevant issues, effectively identifying sanitized taint situations in applications [34]. Zhong and others have constructed system dependence graphs (SDG) for software versions before and after repairs, revealing control dependencies, data dependencies, and call relationships in different processes. By comparing and matching the SDGs of the software versions before and after repairs using graph theory algorithms, the subgraphs of the differences are constructed as PSBPs, enhancing the capability of defect mining [35]. The methods of Higo, Chen, and Zhong have all discovered multiple previously unknown bugs in actual projects, demonstrating the effectiveness of these methods.

However, these methods also have some issues. First, although these repair patterns have made good use of statement dependencies and some semantic information, they are still difficult to mine deep semantic information of the code and eliminate noise in changes by combining context information. Second, the process of mining code defect repair patterns usually relies on program analysis frameworks for specific programming languages. When the analysis capability needs to be ported to other programming languages, it often requires replacing the upstream program analysis framework, significantly increasing the cost of technology transfer. Furthermore, these methods usually independently implement pattern matching and detection engines, requiring more expertise to debug rules.

7. Limitations

- **Dependency and Generalization Ability of LLMs**: This method heavily relies on LLM to generate rule descriptions and static analysis rules based on code repairs. The programming languages it can cover are limited by the scope and quality of the training data of the LLM itself. Even if the static analysis tool supports checking the language, insufficient training on the syntactic features of that programming language may lead to difficulties in parsing the rules.
- **Sample Limitation in the Verification Stage**: Rule verification depends on a limited number of test code snippets. Although our method can generate more code snippets to test the rules, it may still fail to cover all potential boundary conditions.
- **Human Intervention and Efficiency Impact**: Rule optimization requires multiple iterations of adjusting prompts and manual work involvement. In our experiments, each iteration took 10 to 30 minutes, which may limit the rapid generation of large-scale rule sets.
- **Potential Bias in Cross-Language or Cross-Framework Migration**: The rule migration stage relies on natural language descriptions of the rules being migrated as an intermediate representation. If the syntax difference between the source language and the target language is significant, it may lead to errors in the generated natural language descriptions. For example, the type systems of Java and C are different. When migrating rules from Java to C, the generated rule descriptions for C may contain ”matching

variables of type *string*," which may confuse the LLM because C uses *char** as the type of strings.

8. Conclusion

This study presents an LLM-based framework for multi-language static analysis rule generation and optimization, which lowers the expertise threshold for rule customization and enables developers with limited static-analysis knowledge background to create production-ready analysis rules. Our proposed method systematically addresses three core challenges in custom static analysis: (1) corner case coverage through structured prompt engineering, (2) cross-language compatibility via natural-language-mediated rule translation, and (3) computational efficiency via hybrid LLM/static-analysis orchestration. Our methodology introduces three key innovations: First, a hierarchical prompting mechanism that decomposes rule creation into rule-description generation, rule validation, and rule enhancement for failed cases, effectively guiding LLMs to produce analyzer-ready rules. Second, a cross-language migration method with the media of natural language descriptions ensures content consistency across languages of the rules. Third, a resource-aware execution pipeline that combines the LLMs and lightweight static analyzers to improve the efficiency of code scan. Experimental validation demonstrates significant improvements over conventional approaches: In our evaluation on the Semgrep official ruleset, 81.98% of the generated rules were grammatically correct, with 74.73% of them being effective; the grammatically correct rate of cross-language migration rules reached 79.17%, and 84.21% of them were effective; in addition, our method identified 7 potential issues in the Semgrep official rule set and successfully corrected 5 of them. Evaluating on the real-world dataset, our method modestly outperformed the SOTA method in the effectiveness of rule generation, and showed ability by spotting 9 previously unknown bugs on the Linux Kernel. Besides, our method is also capable of identifying and rectifying latent defects in existing rule sets, providing solutions for industrial static analysis maintenance. The future work includes incorporating more learning methodologies and trying to apply it on various of real-world codebases to evaluate the effectiveness of our method.

References

[1] M. Beller, R. Bholanath, S. McIntosh, and A. Zaidman. 2016. Analyzing the State of Static Analysis: A Large-Scale Evaluation in Open Source Software. In 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER), Vol. 1. 470–481. https://doi.org/10.1109/SANER.2016. 105

[2] Mehrpour, S. and LaToza, T.D., 2023. Can static analysis tools find more defects? a qualitative study of design rule violations found by code review. Empirical Software Engineering, 28(1), p.5.

[3] Mendonça, D.S. and Kalinowski, M., 2022. An empirical investigation on the challenges of creating custom static analysis rules for defect localization. Software Quality Journal, 30(3), pp.781-808.

[4] Tymchuk, Y., Ghafari, M. and Nierstrasz, O., 2018, May. JIT feedback: What experienced developers like about static analysis. In Proceedings of the 26th Conference on Program Comprehension (pp. 64-73).

[5] Yang, C., Zhao, Z., Xie, Z., Li, H. and Zhang, L., 2025. KNighter: Transforming Static Analysis with LLM-Synthesized Checkers. arXiv preprint arXiv:2503.09002.

[6] Liang, G.T. and Wang, Q.X., 2012. CODAS: An Extensible Static Code Defect Analysis Service. Computer Science, 39(1).

[7] Vassallo, C., Panichella, S., Palomba, F., Proksch, S., Gall, H.C. and Zaidman, A., 2020. How developers engage with static analysis tools in different contexts. Empirical Software Engineering, 25, pp.1419-1457.

[8] Xue, Z., Gao, Z., Hu, X. and Li, S., 2023, September. ACWRecommender: A Tool for Validating Actionable Warnings with Weak Supervision. In 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE) (pp. 1876-1880). IEEE.

[9] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback.

[10] Mondal, R., Tang, A., Beckett, R., Millstein, T. and Varghese, G., 2023, November. What do LLMs need to synthesize correct router configurations?. In Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (pp. 189-195).

[11] Sharma, P. and Yegneswaran, V., 2023, November. Prosper: Extracting protocol specifications using large language models. In Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (pp. 41-47).

[12] Wang, C., Scazzariello, M., Farshin, A., Ferlin, S., Kostić, D. and Chiesa, M., 2024. Netconfeval: Can llms facilitate network configuration?. Proceedings of the ACM on Networking, 2(CoNEXT2), pp.1-25.

[13] Wang, B., Wang, Z., Wang, X., Cao, Y., A Saurous, R. and Kim, Y., 2023. Grammar prompting for domain-specific language generation with large language models. Advances in Neural Information Processing Systems, 36, pp.65030-65055.

[14] M. Mosthaf, M. and Wasowski, A., 2024, September. From a Natural to a Formal Language with DSL Assistant. In Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (pp. 541-549).

[15] Lamas, V., R. Luaces, M. and Garcia-Gonzalez, D., 2024, September. DSL-Xpert: LLM-driven Generic DSL Code Generation. In Proceedings of the ACM/IEEE 27th

International Conference on Model Driven Engineering Languages and Systems (pp. 16-20).

[16] Pei, K., Bieber, D., Shi, K., Sutton, C. and Yin, P., 2023, July. Can large language models reason about program invariants?. In International Conference on Machine Learning (pp. 27496-27520). PMLR.

[17] Hassan, M., Ahmadi-Pour, S., Qayyum, K., Jha, C.K. and Drechsler, R., 2024, March. Llm-guided formal verification coupled with mutation testing. In 2024 Design, Automation & Test in Europe Conference & Exhibition (DATE) (pp. 1-2). IEEE.

[18] Chen, J., Deng, L., QIU, Y., ZHAO, P., SONG, J. and WANG, X., Llm-Based Automated Modeling in Symbolic Execution for Securing Medical Software. Jingcheng and WANG, Xiaopei, Llm-Based Automated Modeling in Symbolic Execution for Securing Medical Software.

[19] Li, H., Hao, Y., Zhai, Y. and Qian, Z., 2024. Enhancing static analysis for practical bug detection: An llm-integrated approach. Proceedings of the ACM on Programming Languages, 8(OOPSLA1), pp.474-499.

[20] Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X. and Zhang, X., 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948.

[21] Jin, C. and Rinard, M., 2024, July. Emergent representations of program semantics in language models trained on programs. In Forty-first International Conference on Machine Learning.

[22] Korinek, A., 2023. Language models and cognitive automation for economic research (No. w30957). national Bureau of economic Research.

[23] Yang, A.Z., Le Goues, C., Martins, R. and Hellendoorn, V., 2024, February. Large language models for test-free fault localization. In Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (pp. 1-12).

[24] Hossain, S.B., Jiang, N., Zhou, Q., Li, X., Chiang, W.H., Lyu, Y., Nguyen, H. and Tripp, O., 2024. A deep dive into large language models for automated bug localization and repair. Proceedings of the ACM on Software Engineering, 1(FSE), pp.1471-1493.

[25] Yang, H., Zhou, Y., Liang, T. and Kuang, L., 2025. ChatDL: An LLM-Based Defect Localization Approach for Software in IIoT Flexible Manufacturing. IEEE Internet of Things Journal.

[26] Qin, Y., Wang, S., Lou, Y., Dong, J., Wang, K., Li, X. and Mao, X., 2024. Agentfl: Scaling llm-based fault localization to project-level context. arXiv preprint arXiv:2403.16362.

[27] Kang, S., An, G. and Yoo, S., 2024. A quantitative and qualitative evaluation of LLM-based explainable fault localization. Proceedings of the ACM on Software Engineering, 1(FSE), pp.1424-1446.

[28] Yerramreddy, S., Mordahl, A., Koc, U., Wei, S., Foster, J.S., Carpuat, M. and Porter, A.A., 2023. An empirical assessment of machine learning approaches for triaging reports of static analysis tools. Empirical Software Engineering, 28(2), p.28.

[29] Yang, M., Yang, S. and Wong, W.E., 2024. Multi-objective Software Defect Prediction via Multi-source Uncertain Information Fusion and Multi-task Multi-view Learning. IEEE Transactions on Software Engineering.

[30] Chapman, P.J., Rubio-González, C. and Thakur, A.V., 2024, June. Interleaving static analysis and llm prompting. In Proceedings of the 13th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (pp. 9-17).

[31] Johnson B, Song Y, Murphy-Hill E, Bowdidge R (2013) Why don't software developers use static analysis tools to find bugs?In: Proceedings of the 35th International Conference on Software Engineering, pp 672-681

[32] Ray, B., Nagappan, M., Bird, C., Nagappan, N. and Zimmermann, T., 2015, May. The uniqueness of changes: Characteristics and applications. In 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories (pp. 34-44). IEEE.

[33] Higo, Y., Hayashi, S., Hata, H. and Nagappan, M., 2020. Ammonia: an approach for deriving project-specific bug patterns. Empirical Software Engineering, 25(3), pp.1951-1979.

[34] Chen, X., Li, Q., Yang, Z., Liu, Y., Shi, S., Xie, C. and Wen, W., 2021, October. Vulchecker: achieving more effective taint analysis by identifying sanitizers automatically. In 2021 IEEE 20th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) (pp. 774-782). IEEE.

[35] Zhong, H., Wang, X. and Mei, H., 2020. Inferring bug signatures to detect real bugs. IEEE Transactions on Software Engineering, 48(2), pp.571-584.

[36] Sun, C. and Khoo, S.C., 2013, August. Mining succinct predicated bug signatures. In Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (pp. 576-586).

[37] Dolcetti, G., Arceri, V., Iotti, E., Maffeis, S., Cortesi, A. and Zaffanella, E., 2024. Helping LLMs Improve Code Generation Using Feedback from Testing and Static Analysis. arXiv preprint arXiv:2412.14841.

[38] Zheng, T., Liu, H., Xu, H., Chen, X., Yi, P. and Wu, Y., 2024. Few-VulD: A Few-shot learning framework for software vulnerability detection. Computers & Security, 144, p.103992.